



本小节内容

2017 考研真题的 44 题解析

2017 考研真题的 44 题解析

44.(10 分) 在按字节编址的计算机 M 上, 题 43 中 f1 的部分源程序(阴影部分) 与对应的机器级代码 (包括指令的虚拟地址) 如下:

```
int f1( unsigned n)
1    00401020    55          push ebp
    .....
    for( unsigned i=0; i<= n-1; i++)
    .....
20    0040105E    39 4D F4    cmp dword ptr [ebp-0Ch],ecx
    .....
    power *= 2;
    .....
23    00401066    D1 E2    shl  edx,1
    .....
    return sum;
    .....
35    0040107F    C3          ret
```

(1)计算机 M 是 RISC 还是 CISC? 为什么?

(2)f1 的机器指令代码共占多少字节? 要求给出计算过程。

(3)第 20 条指令 cmp 通过 i 减 n-1 实现对 i 和 n-1 的比较。执行 f1(0)过程中当 i=0 时, cmp 指令执行后, 进/借位标志 CF 的内容是什么? 要求给出计算过程。

(4)第 23 条指令 shl 通过左移操作实现了 power *2 运算, 在 f2 中能否也用 shl 指令实现 power *2? 为什么?

答案解析:

1) M 为 CISC。 (1 分)

M 的指令长短不一, 不符合 RISC 指令系统特点。 (1 分)

2) f1 的机器代码占 96B。 (1 分)

因为 f1 的第一条指令“push ebp”所在的虚拟地址为 0040 1020H, 最后一条指令“ret”所在的虚拟地址为 0040 107FH, 所以, f1 的机器指令代码长度为 0040 107FH - 0040 1020H + 1 = 60H = 96 字节。 (1 分)

3) CF = 1。 (1 分)

cmp 指令实现 i 与 n-1 的比较功能, 进行的是减法运算。在执行 f1(0)过程中, n = 0, 当 i = 0 时, i = 0000 0000H, 且 n-1 = FFFF FFFFH。因此, 当执行第 20 条指令时, 在补码加/减运算器中执行“0 减 FFFF FFFFH”的操作, 即 0000 0000H + 0000 0000H + 1 = 0000 0001H, 此时进位输出 C = 0, 减法运算时的借位标志 CF = C^1, 即 C 异或 1, 因此借位标志 CF 为 1。 (2 分)



可能你会问减法为什么 借位为什么是进位异或上 1 呢？我们举个例子

以 1 个字节来看

例如 3-8, 实际是 $3 + (-8)$

```
0000 0011  3
1111 1000 -8
```

直接相加得到的结果是 1111 1011 没有产生进位, 但是 3-8 实际要借位的, 因此 CF=进位 C 异或 1, CF 是 1, 也就是 3 是小于 8 的

例如 3-2, 实际是 $3 + (-2)$

```
0000 0011  3
1111 1110 (-2)
```

0000 0001 得到结果, 并产生了进位, 也就是 C=1, 这时 CF=进位 C 异或 1, 所以 CF 值为 0, 没有产生借位, 3 大于 2

其实我们直接用正常的数学就能判断出进借位标志值, 而大题是要求我们描述这个计算机的运算过程。

4) f2 中不能用 shl 指令实现 $power*2$ 。(1 分)

因为 shl 指令用来将一个整数的所有有效数位作为一个整体左移; 而 f2 中的变量 power 是 float 型, 其机器数中不包含最高有效数位, 但包含了阶码部分, 将其作为一个整体左移时并不能实现“乘 2”的功能, 因而 f2 中不能用 shl 指令实现 $power*2$ 。(2 分) 浮点数运算比整型运算要复杂, 耗时也较长。

下面附带一下考研的 C 代码及对应汇编。

```
int f1(unsigned n)
{
    int sum=1,power=1;

    for(unsigned i=0;i<n-1;i++)
    {
        power*=2;

        sum+=power;
    }
}
```



```
    }  
  
    return sum;  
  
}  
  
int main() {  
  
    f1(10);  
  
    return 0;  
  
}
```

下面是汇编代码，和考研的汇编有一点点差异（我的是 AMD 的 cpu，

movl \$0x1,-0x4(%ebp)是把 1 放到 ebp 减 4 的地方），但是原理全部是一致的。

```
int f1(unsigned n)  
{  
    40150f: 90                nop  
  
00401510 <_f1>:  
    401510: 55                push    %ebp  
    401511: 89 e5             mov     %esp,%ebp  
    401513: 83 ec 10          sub     $0x10,%esp  
  
    int sum=1,power=1;  
    401516: c7 45 fc 01 00 00 00 movl    $0x1,-0x4(%ebp)  
    40151d: c7 45 f8 01 00 00 00 movl    $0x1,-0x8(%ebp)  
  
    for(unsigned i=0;i<n-1;i++)  
    401524: c7 45 f4 00 00 00 00 movl    $0x0,-0xc(%ebp)  
    40152b: eb 0d             jmp     40153a <_f1+0x2a>  
  
    {  
        power*=2;  
    40152d: d1 65 f8          shll    -0x8(%ebp)  
  
        sum+=power;  
    401530: 8b 45 f8          mov     -0x8(%ebp),%eax  
    401533: 01 45 fc          add     %eax,-0x4(%ebp)  
  
        for(unsigned i=0;i<n-1;i++)  
    401536: 83 45 f4 01       addl    $0x1,-0xc(%ebp)  
    40153a: 8b 45 08          mov     0x8(%ebp),%eax  
    40153d: 83 e8 01          sub     $0x1,%eax  
    401540: 39 45 f4          cmp     %eax,-0xc(%ebp)
```



```
401543: 72 e8          jb     40152d <_f1+0x1d>
    }
    return sum;
401545: 8b 45 fc       mov     -0x4(%ebp),%eax
    }
401548: c9            leave
401549: c3            ret
```

0040154a <_main>:

```
int main() {
40154a: 55            push    %ebp
40154b: 89 e5         mov     %esp,%ebp
40154d: 83 e4 f0     and     $0xfffffff0,%esp
401550: 83 ec 10     sub     $0x10,%esp
401553: e8 a8 00 00 00 call    401600 <__main>
    f1(10);
401558: c7 04 24 0a 00 00 00 movl    $0xa,(%esp)
40155f: e8 ac ff ff ff call    401510 <_f1>
    return 0;
401564: b8 00 00 00 00 mov     $0x0,%eax
401569: c9            leave
40156a: c3            ret
40156b: 90            nop
40156c: 66 90        xchg    %ax,%ax
40156e: 66 90        xchg    %ax,%ax
```